

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 3 Abgabe KW 19

1. Aufgabe (Codeopolis)

Die Klasse `DomainModel.Plants.Conditions` ist inhaltlich sehr eng mit der Klasse `Grain` verwandt. Es gibt daher Argumente, sie als innere Klasse zu implementieren. Refactorieren Sie daher die Implementierung so, dass `Conditions` eine statische innere Klasse von `Grain` ist. Überlegen Sie, welche Vor- und Nachteile dies hat und diskutieren Sie dies im Review.

2. Aufgabe (Codeopolis)

Die Klasse `Depot` verwaltet Silos mit verschiedenen Getreidesorten. Um es zu ermöglichen, nur über die Silos des Depots zu iterieren, die eine bestimmte Getreidesorte enthalten, soll ein entsprechender Iterator hinzugefügt werden. Da der Iterator nur im Kontext der Klasse `Depot` sinnvoll ist, soll er als Inner Class implementiert werden. Gehen Sie bei der Implementierung wie folgt vor:

(a) Implementieren Sie in der Klasse `Depot` das folgende innere Interface:

```
1 public interface Iterator {
2     /**
3      * Checks if there are further objects available for iteration.
4      *
5      * @return {@code true} if more objects are available; {@code false}
6      *         otherwise.
7      */
8     boolean hasNext();
9
10    /**
11     * Returns the next {@link Silo.Status} object in the iteration.
12     * This method should only be called if {@code hasNext()} returns {
13     *     {@code true}.
14     *
15     * @return The next {@link Silo.Status} object.
16     * @throws NoSuchElementException if no more elements are available
17     */
18     Silo.Status next();
19 }
```

- (b) Implementieren Sie die Klasse `DepotIterator` als private innere Klasse, die das Interface `Iterator` implementiert. Über den Konstruktor des Iterators wird die Getreidesorte gesetzt. Der Iterator iteriert dann nur über die Silos dieses Typs.
- (c) Der Iterator gibt ein Objekt vom Typ `Silo.Status` zurück. Implementieren Sie diesen Typ als innere Klasse. Die Klasse `Silo.Status` soll public sein, aber nur private Konstruktoren besitzen, so dass sie nur innerhalb der Klasse `Silo` instanziiert werden kann. `Silo.Status` speichert die Kapazität und den aktuellen Füllstand des Silos. Stellen Sie sicher, dass Objekte der Klasse `Silo.Status` immutable¹ sind.
Um ein `Silo.Status`-Objekt erzeugen zu können, fügen Sie der `Silo`-Klasse eine `getStatus`-Methode hinzu.
- (d) Implementieren Sie eine public-Methode, die ein Iterator-Objekt zurückgibt, damit ein Iterator-Objekt auch außerhalb der `Depot`-Klasse instanziiert werden kann.

Schreiben Sie die Methoden `public int getCapacity(Game.GrainType grainType)` und `public int getFillLevel(Game.GrainType grainType)` so um, dass der Iterator verwendet wird.

Überlegen Sie, wie Sie diese Funktionalität ohne innere Klassen implementieren können und diskutieren Sie im Review die Vor- und Nachteile der Implementierungsvarianten.

3. Aufgabe

Implementieren Sie eine Klasse `NumberProcessor` wie folgt:

```

1 public class NumberProcessor {
2     // Enthält die zu verarbeitenden Werte
3     private int[] values;
4
5     /**
6      * Konstruktor erhält das zu analysierende Array.
7      * @param values das int-Array, z. B. Messwerte oder Punktestände
8      */
9     public NumberProcessor(int[] values) {
10         this.values = values.clone(); // Kopie für Immutability
11     }
12 }
```

Fügen Sie der Klasse eine Methode `public void printGreaterThan(int threshold)` hinzu. Diese Methode gibt alle Werte des `values`-Arrays aus, die größer als `threshold` sind. Verwenden Sie eine lokale Klasse, um das Array zu filtern.

Überlegen Sie welche Vor- und Nachteile die Implementierung mit einer lokalen Klasse hat und diskutieren Sie diese im Review.

¹Objekte sind immutable, wenn ihre Werte nach der Erzeugung nicht mehr geändert werden können. Es ist oft sehr vorteilhaft, solche Objekte zu implementieren, da der Code dadurch zuverlässiger wird. Siehe auch <https://docs.oracle.com/javase/tutorial/essential/concurrency/immutable.html>.

4. Aufgabe

Gegeben sei das folgende Interface:

```

1 public interface BinaryOperation {
2     /**
3      * Wendet eine Operation auf a und b an.
4      * @param a erster Operand
5      * @param b zweiter Operand
6      * @return Ergebnis der Operation
7      */
8     int apply(int a, int b);
9 }
    
```

Implementieren Sie eine Klasse Calculator, mit einer Methode compute, die wie folgt definiert ist:

```

1 /**
2  * Wendet eine übergebene Operation op auf a und b an.
3  * @param a erster Operand
4  * @param b zweiter Operand
5  * @param op Operation, die auf die beiden Operanden angewendet wird
6  * @return Ergebnis der Operation
7  */
8 public int compute(int a, int b, BinaryOperation op) {}
    
```

Instanzieren Sie nun in einer main-Methode ein Objekt der Klasse Calculator und implementieren Sie vier Aufrufe von compute(x, y, ...), bei denen Sie jeweils eine anonyme BinaryOperation-Klasse übergeben, die eine Addition, Subtraktion, Multiplikation bzw. Ganzzahldivision durchführt.

Überlegen Sie, warum an dieser Stelle eine Implementierung mit anonymen Klassen sinnvoll ist und diskutieren Sie dies im Review.

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... in der Lage sein, die verschiedenen Typen von inneren Klassen zu implementieren.
- ... die Eigenschaften und Unterschiede der Typen innerer Klassen erklären können.
- ... die Vor- und Nachteile der Implementierung mit inneren Klassen und mit Top-Level-Klassen erläutern können.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe (Codeopolis)

Die Objekte der Klasse `TurnResult` werden nur innerhalb der `City` erzeugt. Um sicherzustellen, dass dies auch nur dort möglich ist, ist es sinnvoll dies als innere Klasse zu implementieren. Refaktorisieren Sie die Implementierung entsprechend.

Überlegen Sie, welche Vor- und Nachteile diese Implementierung hat und diskutieren Sie dies im Review.

2. Aufgabe (Codeopolis)

Schreiben Sie die Methode `Depot.toString()` so um, dass eine lokale Klasse `DepotVisualizer` verwendet wird, um die String-Repräsentation eines Silos zu erzeugen. Die eigentliche `toString`-Methode sieht dann wie folgt aus:

```
DepotVisualizer result = new DepotVisualizer();
for (Silo silo : silos) {
    result.appendSiloInfo(silo);
}
return result.visualize();
```

Überlegen Sie welche Vor- und Nachteile die Implementierung mittels einer lokalen Klasse hat und diskutieren Sie diese im Review.

3. Aufgabe (Codeopolis)

Passen Sie die Methode `City.plant` so an, dass beim Pflanzen von Weizen mit einer Wahrscheinlichkeit von 0.1 eine spezielle Weizensorte gepflanzt wird, die den doppelten Grundertrag hat. Implementieren Sie diese spezielle Sorte als anonyme Klasse.

Überlegen Sie, warum es sinnvoll ist, dies als anonyme Klasse zu implementieren und diskutieren Sie dies im Review.

4. Aufgabe

Fügen Sie der Klasse `NumberProcessor` folgende Methoden hinzu und nutzen Sie bei der Implementierung der Methoden lokale Klassen:

- `public void printInRange(int min, int max)`: Gibt alle Werte aus, die im Bereich `[min, max]` liegen. Definieren Sie dazu eine lokale Klasse `RangeFilter`.
- `public void printEvenNumbers()`: Gibt alle geraden Werte aus. Definieren Sie zum Filtern der Werte eine lokale Klasse `EvenFilter`.
- `public void printStatistics()`: Gibt Summe, Durchschnitt, Minimum und Maximum der Werte aus. Definieren Sie eine lokale Klasse `Stats`, die Methoden `getSum()`, `getAverage()`, `getMin()`, `getMax()` bereitstellt.